MODELARE ȘI SIMULARE

Săptămâna 5. Simularea cu evenimente discrete

Suport de curs · Anul III, Calculatoare și Tehnologia Informației
Ș.l. dr. ing. Cătălin Iapă

Obiective

1. să explicați noțiunile de stare, eveniment, ceas simulat și listă a evenimentelor viitoare;
2. să comparați simularea cu pas fix (time-stepping) cu simularea condusă de evenimente (event-driven);
3. să urmăriți de mână o simulare și să calculați corect medii ponderate în timp și medii pe clienți;
4. să scrieți un motor de simulare cu evenimente discrete în Python și să-l validați pe coada M/M/1;
5. să descrieți cele trei abordări clasice: programarea evenimentelor, scanarea activităților, interacțiunea proceselor;
6. să modelați în SimPy sisteme cu resurse: o intersecție, o linie de producție, o unitate de primiri urgențe.

Cuvinte cheie: *simulare cu evenimente discrete, eveniment, stare, ceas simulat, lista evenimentelor viitoare, heap, time-stepping, event-driven, medie ponderată în timp, graful evenimentelor, graful activităților, event scheduling, activity scanning, process interaction, SimPy, resursă, proces, buffer, prioritate, simulare paralelă, Time Warp*

1. Stare, eveniment, ceas

1.1. Ce este simularea cu evenimente discrete

Formulele din săptămâna a patra sunt valabile doar în condiții restrictive: sosiri Poisson, adesea servire exponențială și regim staționar. Sistemele reale au însă vârfuri de trafic, priorități, clienți care renunță, resurse care se defectează și programe de lucru variabile, iar managerii întreabă adesea despre regimul tranzitoriu sau despre percentile. În aceste situații construim un **program care imită sistemul** și măsurăm în el mărimile care ne interesează.

Într-o **simulare cu evenimente discrete** (DES), starea sistemului se schimbă doar în anumite momente, numite **evenimente**: sosește un client, se termină o servire, un semafor trece pe verde. Între două evenimente consecutive starea rămâne constantă, așa că simulatorul **sare** direct de la un eveniment la următorul. Timpul simulat este continuu, iar costul simulării depinde de numărul de evenimente, nu de lungimea intervalului simulat. De exemplu, o zi la o bancă, cu 300 de clienți, înseamnă aproximativ 600 de evenimente; cu un pas fix de o secundă ar fi fost 28 800 de pași, aproape toți fără nicio schimbare de stare.

Vocabularul de bază: **entitatea** circulă prin sistem (client, pachet, piesă, pacient) și are **atribute** (momentul sosirii, prioritatea); **starea** conține tot ce trebuie să știm pentru a continua simularea; un **eveniment** schimbă starea într-un moment, iar o **activitate** durează un interval; **procesul** este succesiunea de evenimente și activități a unei entități; **resursa** este folosită și eliberată (casier, CPU, medic), iar **coada** conține entitățile care așteaptă o resursă.

1.2. Time-stepping vs event-driven: bila în cutie (Cernazean)

Profesorul Cernazean ilustrează diferența printr-un exemplu simplu: o particulă se mișcă fără forțe într-o cutie de 16 m × 16 m, pornind din (4, 5) cu viteza (2; 0,9) m/s, și ricoșează perfect elastic din pereți. Abordarea

cu pas fix avansează timpul cu Δt, verifică pereții și mută bila; ciocnirea este detectată abia după ce bila a trecut de perete, deci fiecare ciocnire introduce o mică eroare. Abordarea **condusă de evenimente** calculează direct momentul următoarei ciocniri: peretele de sus ar fi atins după (16 − 5)/0,9 = 12,22 s, cel din dreapta după (16 − 4)/2 = 6 s, iar ceilalți doi pereți dau timpi negativi; următorul eveniment este minimul pozitiv, 6 s. În 180 de secunde au loc 32 de ciocniri (22 cu pereții verticali și 10 cu cei orizontali), iar poziția finală (12; 7) este exactă (Figura 1).

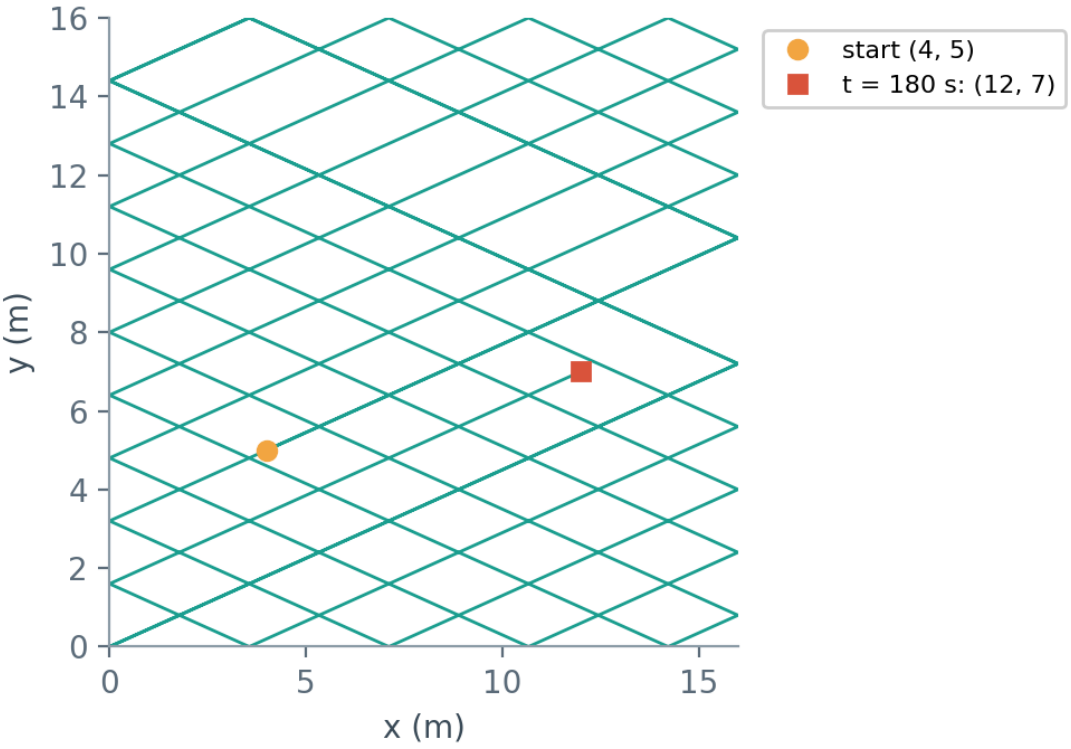


Figura 1. Traiectoria bilei timp de 3 minute, calculată cu 32 de evenimente (ciocniri cu pereții)

Tabelul 1. Pas fix vs evenimente pentru bila în cutie (t = 180 s)

Metoda	Iterații	Eroarea finală (m)
pas fix, Δt = 0,1	1 800	4,535
pas fix, Δt = 0,01	18 000	0,454
pas fix, Δt = 0,001	180 000	0,045
evenimente	32	0 (exact)

Cu pas fix, eroarea scade doar proporțional cu Δt, deci o precizie de zece ori mai mare costă de zece ori mai mulți pași. Regula practică: dacă putem calcula **când** se schimbă starea, folosim evenimente; dacă starea se schimbă tot timpul, sub acțiunea unor forțe continue, folosim pas fix și metodele numerice din săptămâna a șaptea. Atenție la terminologie: într-o simulare DES timpul este continuu, iar într-o simulare a unui model continuu timpul este de fapt discretizat.

1.3. Anatomia unui simulator

Un eveniment are un moment, un tip și eventual date suplimentare. Fiecărui tip de eveniment îi corespunde o **acțiune**, adică o funcție care modifică starea S și **lista evenimentelor viitoare** Q: a: (S, Q) → (S, Q). Acțiunea

poate programa evenimente noi sau poate anula evenimente programate. Evenimentele **endogene** sunt consecințe ale evoluției sistemului (terminarea unei serviri), iar cele **exogene** vin din exterior (sosirile) și pot fi generate dinainte, din date sau pseudo-aleator.

Ceasul simulat indică momentul curent din lumea modelului. **Lista evenimentelor viitoare** (FEL) conține toate evenimentele programate, ordonate după moment – o coadă de priorități. Algoritmul este același pentru orice model: inițializăm ceasul, starea și primele evenimente; apoi, cât timp nu este îndeplinită condiția de oprire, extragem evenimentul cu cel mai mic moment, mutăm ceasul la acel moment și executăm acțiunea, care modifică starea, programează evenimente și colectează statistici. Condiția de oprire poate fi un orizont de timp, un număr de entități, lista goală, un eveniment special de sfârșit sau o precizie statistică.

```
t = t0
Q = evenimente_exogene()      # de exemplu, sosirile
S = stare_initiala(t0)
while not conditie_oprire(t, S, Q):
    e = Q.extrage_minim()      # următorul eveniment
    t = e.moment               # sărim la el
    S, Q = e.actiune(S, Q)     # schimbă starea și lista
```

Pentru o coadă cu un server ajung două tipuri de evenimente. La o **sosire**: n crește cu 1, se programează sosirea următoare, iar dacă serverul era liber începe servirea și se programează plecarea; altfel clientul intră în coadă. La o **plecare**: n scade cu 1, serverul se eliberează, iar dacă cineva așteaptă, începe servirea lui și se programează plecarea lui. Logica nu depinde de distribuții; pentru alt model schimbăm doar generatoarele de timpi. Schruben (1983) a propus desenarea acestei logici ca **graf al evenimentelor**: nodurile sunt tipuri de evenimente, iar arcele înseamnă „programează, după o întârziere, dacă o condiție este adevărată”.

1.4. O urmă de mână și două feluri de medii

Cinci clienți sosesc la momentele 1, 2, 5, 8 și 9, cu timpii de servire 3, 2, 1, 2 și 1,5. Urmărind evenimentele în ordine, obținem așteptările 0, 2, 1, 0 și 1 și timpii în sistem 3, 4, 2, 2 și 2,5; ultimul client pleacă la 11,5 (Figura 2). Numărul mediu de clienți este o **medie ponderată în timp**: aria de sub N(t) împărțită la durată, $\bar{L}$ = 13,5/11,5 = 1,17; utilizarea este $\hat{\rho}$ = 9,5/11,5 = 0,83. Timpul mediu în sistem este o **medie pe clienți**: $\bar{W}$ = 2,7. Legea lui Little leagă cele două medii: $\hat{\lambda}\cdot\bar{W}$ = (5/11,5)·2,7 = 1,17. O greșeală frecventă este media simplă a valorilor lui N la evenimente, care ignoră faptul că intervalele au durate diferite.

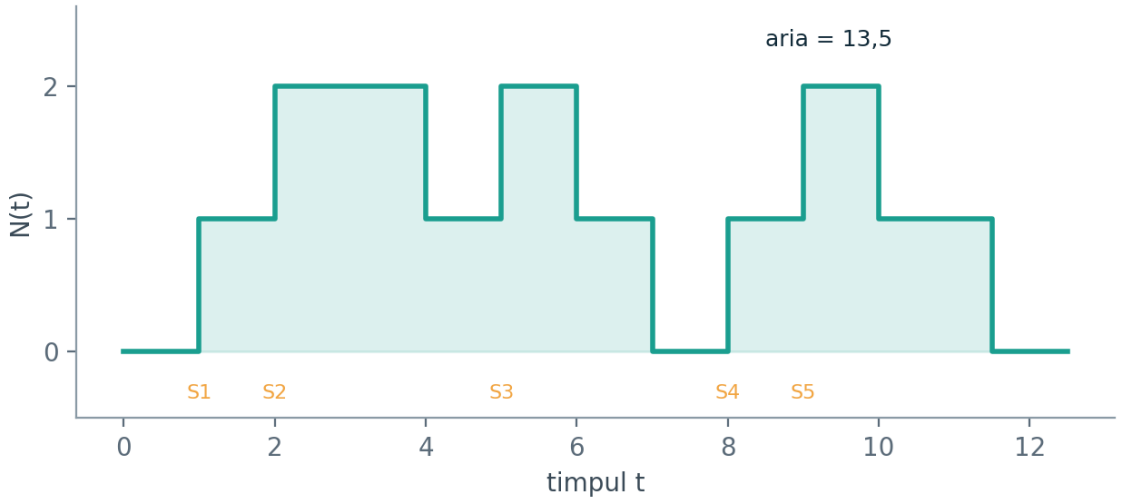


Figura 2. Numărul de clienți N(t) pentru urma de mână; aria de sub curbă este 13,5

1.5. Graful activităților (Holban)

Profesorul Holban descrie sistemele prin **graful activităților**, cu blocuri pentru sosire, așteptare, servire, plecare, alocarea și eliberarea unei resurse dintr-un depozit, iar arcele pot purta probabilități de rutare. În studiul de caz al benzinăriei cu două pompe (sosiri exponențiale la 240 u.t., alimentare N(96; 32) u.t.), simularea dă așteptarea medie, lungimea cozii, încărcarea pompelor, capacitatea de trecere și timpul de răspuns. În varianta a patra, 30% dintre șoferi se opresc la magazin (în medie 23 u.t.), iar 53% dintre ei cumpără; staționarea este o așteptare cu timp asociat, iar rezultatul dimensionează parcarea magazinului. Cernazean folosește un oficiu poștal cu ghișee care se deschid și se închid după program: simularea devine o **funcție obiectiv** pentru optimizarea programului ghișeelor.

De reținut

DES: starea se schimbă doar la evenimente; ceasul sare între ele. Componente: ceas, lista evenimentelor viitoare (coadă de priorități), acțiuni care schimbă starea și lista. Evenimente endogene și exogene. Pas fix vs evenimente: bila în cutie – 32 de evenimente exacte față de zeci de mii de pași aproximativi. Mediile ponderate în timp ($\bar{L}$, ρ) diferă de mediile pe clienți ($\bar{W}$); Little le leagă.

2. Un motor DES de la zero

2.1. Heap-ul și bucla principală

Lista evenimentelor viitoare se implementează cu un **heap binar** (modulul heapq): inserarea și extragerea minimului costă O(log n). Un eveniment este tuplul (moment, număr de ordine, acțiune, argumente); numărul de ordine rezolvă egalitățile – la momente egale câștigă evenimentul programat primul, iar Python nu mai încearcă să compare funcțiile. Motorul ține doar ceasul și lista; modelul ține starea și definește acțiunile.

```
import heapq, itertools

class Sim:
    def __init__(self):
        self.now = 0.0
        self.fel = []                  # lista evenimentelor viitoare
        self.seq = itertools.count()   # departajează evenimentele simultane
    def schedule(self, delay, action, *args):
        heapq.heappush(self.fel, (self.now + delay, next(self.seq), action, args))
    def run(self, until=float("inf")):
        while self.fel and self.fel[0][0] <= until:
            self.now, _, action, args = heapq.heappop(self.fel)
            action(*args)
```

Modelul M/M/1 ține numărul de clienți, starea serverului, coada (momentele de sosire) și acumulatorii statistici: aria de sub N(t), timpul cât serverul a fost ocupat și lista așteptărilor. Metoda update, apelată la începutul fiecărui eveniment, adaugă contribuția intervalului scurs de la evenimentul anterior. Acțiunile arrival, start și departure urmează exact logica din secțiunea 1.3. Codul complet se află în notebook-ul săptămânii.

Tabelul 2. Motorul nostru vs formulele M/M/1 (μ = 1, 200 000 de unități de timp)

ρ	L simulat	L teoretic	W simulat (Little)	W teoretic
0,5	0,99	1,00	1,98	2,00
0,8	4,05	4,00	5,06	5,00
0,9	9,50	9,00	10,56	10,00

La ρ = 0,5 și 0,8 simularea reproduce formulele la 1–2%; la ρ = 0,9 diferența este de aproximativ 5%, pentru că la utilizare mare perioadele de congestie sunt lungi și corelate – vom trata această problemă cu replicări și intervale de încredere în săptămâna a șasea. Simularea dă și distribuții complete, nu doar medii: la ρ = 0,8, 20% dintre clienți nu așteaptă deloc, iar percentila 95 a așteptării este 14,2 (teoretic 13,9) (Figura 3). Motorul scris în Python pur procesează aproximativ 1,0 milioane de evenimente pe secundă, deci o zi la bancă se simulează în mai puțin de o milisecundă.

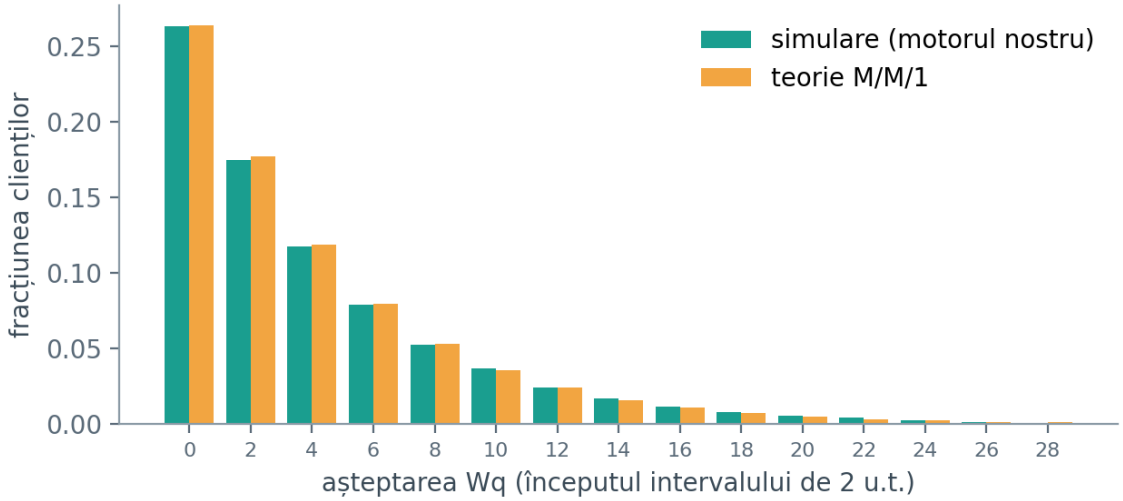


Figura 3. Distribuția așteptării în M/M/1 la ρ = 0,8, pentru clienții care așteaptă: simulare și formula ρ·e^(−(μ−λ)t)

Verificarea unui simulator combină mai multe teste: urme pe cazuri mici, comparate cu calculul de mână; cazuri extreme (λ foarte mic, timpi constanți); conservarea clienților (sosiri = plecări + clienți rămași); legea lui Little pe datele simulate și formulele cunoscute acolo unde există. **Verificarea** stabilește că programul face ce credem noi; **validarea**, adică asemănarea modelului cu sistemul real, este tema săptămânii a șasea.

2.2. Extensii ale modelului

Separarea dintre motor și model face extensiile ușoare. Pentru **M/M/c** numărăm serverele libere în loc de o variabilă booleană; la ρ = 0,9, așteptarea medie simulată este 2,63 cu 3 servere (Erlang C: 2,72) și 0,68 cu 10 servere (Erlang C: 0,67). Pentru **priorități** non-preemptive păstrăm câte o coadă pe clasă; cu 30% cereri interactive și ρ = 0,8, acestea așteaptă 1,06 (formula lui Cobham: 1,05), iar joburile batch 5,14 (5,26); media ponderată rămâne cea de la FIFO, 4 – prioritățile redistribuie așteptarea, nu o elimină. Pentru **clienții care renunță** programăm la sosire și un eveniment de renunțare, care nu mai face nimic dacă clientul a fost între timp servit (anulare „leneșă”); cu λ = 1,2 > μ = 1 și o răbdare medie de 5, 30% dintre clienți renunță, iar sistemul rămâne stabil. Alte extensii: capacitate finită, defecțiuni, program de lucru, rutare în rețele, rate variabile în timp, date reale (simulare condusă de jurnale).

2.3. Trei abordări și câteva detalii de implementare

Există trei moduri clasice de a organiza un simulator. **Programarea evenimentelor** descrie modelul ca tipuri de evenimente (ce am făcut noi); este eficientă, dar logica unei entități se împrăștie în mai multe rutine. **Scanarea activităților** în varianta în trei faze (Tocher, 1963) separă evenimentele B, cu momente cunoscute, de evenimentele C, care au loc când devine adevărată o condiție. **Interacțiunea proceselor** descrie viața fiecărei entități ca un program secvențial care așteaptă; motorul suspendă și reia procesele. Toate trei dau aceleași rezultate. Istoric: GPSS (1961), SIMSCRIPT (1963), Simula 67 – primul limbaj orientat pe obiecte, creat pentru simulare –, SIMAN/Arena, AnyLogic și SimPy (2002).

Structura listei de evenimente contează: o listă sortată are inserarea O(n), un heap O(log n), iar structuri ca calendar queue (Brown, 1988) ajung la O(1) în medie. În testul „hold”, la un milion de evenimente în listă, lista sortată a fost de aproximativ 259 de ori mai lentă decât heap-ul (Figura 4). Capcane frecvente: statistici actualizate după schimbarea stării, evenimente simultane fără o regulă clară, același generator aleator pentru toate sursele (folosiți fluxuri separate), medii simple în loc de medii ponderate în timp, ignorarea perioadei de încălzire și raportarea unei singure rulări.

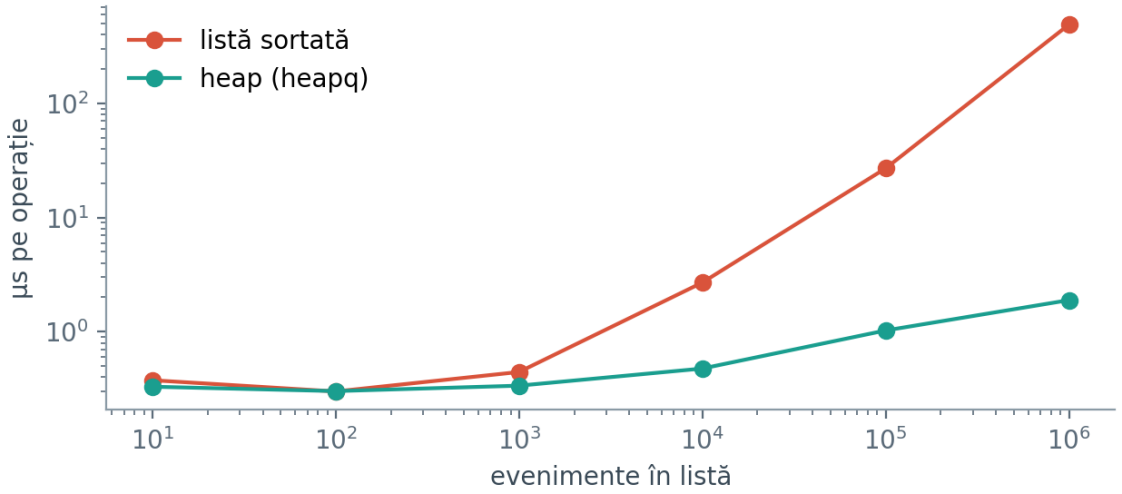


Figura 4. Timpul pe operație pentru lista evenimentelor: listă sortată vs heap (modelul „hold”, scări logaritmice)

De reținut

Motorul DES: heap + buclă „extrage minimul, mută ceasul, execută acțiunea”, în aproximativ 10 linii. Modelul M/M/1 reproduce formulele din S4; extensiile (M/M/c, priorități, renunțări) nu ating motorul. Trei abordări: evenimente, activități (trei faze), procese. Contează structura FEL, ordinea evenimentelor simultane și fluxurile aleatoare separate.

3. SimPy și aplicații

3.1. SimPy

SimPy este o bibliotecă Python open source pentru simulare orientată pe procese. **Environment** ține ceasul și lista evenimentelor, **process** pornește o entitate descrisă ca funcție generator, iar yield suspendă procesul până la un eveniment: env.timeout(d) (trecerea timpului), request() pe o **Resource** sau **PriorityResource**,

get()/put() pe un **Container** sau **Store**. Dedesubt se află tot o listă de evenimente într-un heap, exact ca în motorul nostru.

```
def customer(env):
    t0 = env.now
    with server.request() as req:
        yield req                                  # așteaptă serverul
        yield env.timeout(rng.expovariate(mu))     # servirea
    times.append(env.now - t0)

def source(env):
    while True:
        yield env.timeout(rng.expovariate(lam))
        env.process(customer(env))
```

Pentru M/M/1 cu ρ = 0,8, SimPy dă W = 5,048, motorul nostru 5,060, iar formula 5; SimPy a fost de aproximativ 7 ori mai lent, prețul lizibilității.

3.2. Intersecția cu semafor (Cernazean)

O stradă secundară intră într-o arteră principală printr-un semafor la cerere. Starea este culoarea semaforului f și numărul de mașini care așteaptă C; evenimentele sunt CAR (sosirea unei mașini, exogen), R2G (roșu → verde) și G2R (verde → roșu). Prima mașină care ajunge pe roșu programează R2G după latența a; R2G programează G2R după C·b secunde și eliberează mașinile. Pentru a = 30 s, b = 10 s și mașini la 10, 25, 35, 60 și 75 s, semaforul trece pe verde la 40 (pentru 30 s) și la 105 (pentru 10 s), iar așteptările sunt 30, 15, 5, 0 și 30 s. În codul original, ramura pentru prima mașină pe roșu nu incrementa C; tabelul de urmă din curs era însă corect – un exemplu bun de eroare prinsă prin compararea urmei cu codul.

Cu sosiri Poisson (în medie o mașină la 20 s) și b = 10 s, aproximativ o treime dintre mașini prind verde; așteptarea medie crește aproape liniar cu latența: 5,6 s pentru a = 10 s și 25,3 s pentru a = 60 s (Figura 5). Extensii realiste: limitarea verdelui, timpi de trecere, treceri de pietoni, sincronizarea semafoarelor.

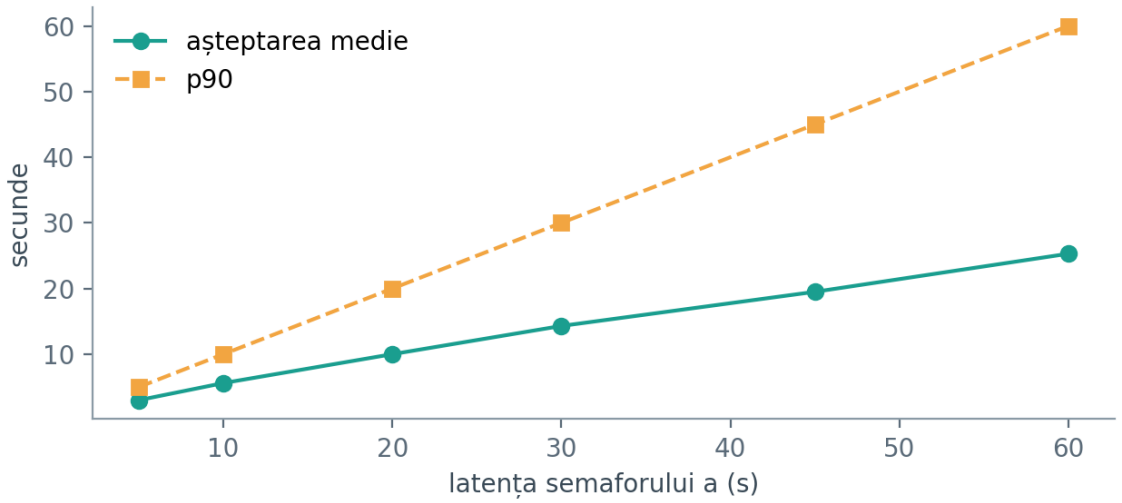


Figura 5. Intersecția: așteptarea medie și percentila 90 în funcție de latența semaforului (b = 10 s)

3.3. Linia de producție

Trei stații în serie, cu timpi medii de 1, 1,1 și 0,9 minute, sunt separate de buffere de capacitate B (Store în SimPy). O stație care nu își poate preda piesa se **blochează**, iar una fără piese se **înfometează**. Stația 2 este gâtul de sticlă, deci throughput-ul nu poate depăși 1/1,1 ≈ 0,91 piese/min. Cu timpi exponențiali și B = 1 linia

produce doar 0,66 piese/min, iar cu B = 20 ajunge la 0,89; cu timpi constanți atinge maximul chiar cu B = 1 (Figura 6). Bufferele există pentru a absorbi variabilitatea, exact lecția ecuației VUT din săptămâna a patra.

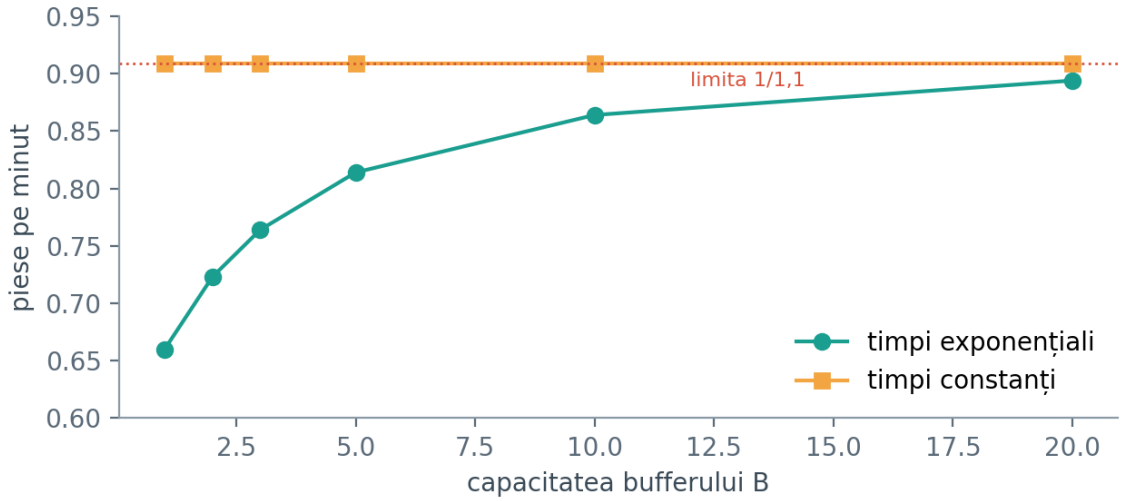


Figura 6. Throughput-ul liniei de producție în funcție de capacitatea bufferelor

3.4. Unitatea de primiri urgențe și parcarea

Pacienții sosesc în medie 6 pe oră, sunt triați în trei priorități (20%, 30%, 50%) și sunt consultați în medie 25 de minute, în ordinea priorității (PriorityResource, non-preemptiv). Încărcarea oferită este 2,5 medici. Cu 3 medici, pacienții de prioritate 3 așteaptă în medie 60 de minute, iar 10% dintre ei peste 172 de minute; al patrulea medic reduce așteptarea medie la 8 minute (Figura 7). Cu priorități și utilizare mare, media poate ascunde așteptări foarte lungi pentru clasele mici – raportăm percentile pe clase.

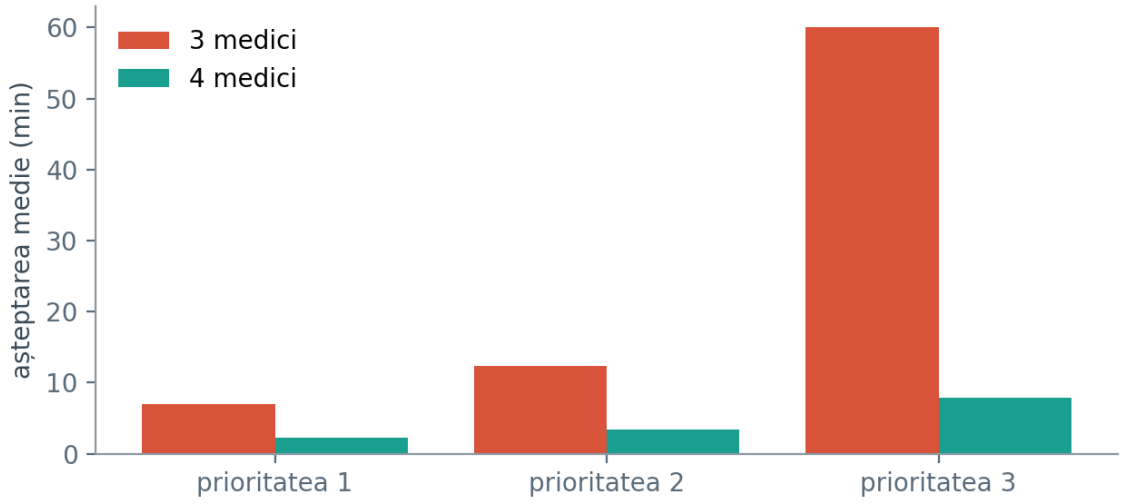


Figura 7. Așteptarea medie până la medic pe clase de prioritate, cu 3 și 4 medici

Studiul de caz al parcării cu 70 de locuri (Holban) arată o lecție de validare: cu datele din curs (sosiri la 240 s, staționare medie 1 000 s), legea lui Little dă a = 1 000/240 ≈ 4,2 locuri ocupate în medie, iar simularea confirmă – ocuparea medie este 4,3, maximul 9, niciun șofer refuzat (Figura 8). Datele sunt probabil transcrise greșit. În varianta aglomerată (sosiri la 40 s, staționare medie 3 000 s, a = 75), simularea dă 12,8% mașini refuzate, foarte aproape de formula Erlang B (13,1%), care depinde doar de durata medie a staționării.



Figura 8. Ocuparea parcării cu 70 de locuri pe o zi de 8 ore, cu datele din studiul de caz

3.5. DES la scară mare

În industrie se folosesc unelte grafice precum Arena, AnyLogic, Simio, FlexSim, Plant Simulation și SimEvents; pentru rețele, ns-3 și OMNeT++. Simularea cu evenimente discrete este nucleul multor **digital twin-uri** de fabrici: modelul pornește din starea curentă a sistemului, citită din date, și simulează înainte variante de planificare. Pentru modele foarte mari, **simularea paralelă** împarte modelul în procese logice care schimbă mesaje cu marcaj de timp. Abordarea conservativă (Chandy–Misra–Bryant) avansează doar când este sigur că nu va primi mesaje din trecut, folosind mesaje nule și lookahead; abordarea optimistă (Time Warp, Jefferson) avansează oricum și, la un mesaj întârziat, revine la o stare salvată și trimite anti-mesaje; memoria se eliberează sub timpul virtual global (GVT).

De reținut

SimPy = procese (generatoare Python) + resurse + timeout-uri, peste o listă de evenimente. Intersecția: stare (f, C), evenimente CAR, R2G, G2R. Linia de producție: blocare, înfometare, gât de sticlă; bufferele absorb variabilitatea. Urgențe: priorități, percentile pe clase. Parcarea: verificați ordinele de mărime (Little) înainte de simulare. La scară mare: digital twins, simulare paralelă conservativă sau optimistă.

Glosar RO – EN

Română	English
simulare cu evenimente discrete	discrete-event simulation (DES)
eveniment	event
stare	state
ceas simulat	simulation clock
lista evenimentelor viitoare	future event list (FEL)
pas fix de timp	time-stepping, fixed time step

Română	English
condus de evenimente	event-driven
eveniment endogen / exogen	endogenous / exogenous event
acțiune	action, event routine
condiție de oprire	stopping condition
medie ponderată în timp	time-weighted average
graful evenimentelor	event graph
graful activităților	activity graph
coadă de priorități	priority queue
heap binar	binary heap
programarea evenimentelor	event scheduling
scanarea activităților	activity scanning
interacțiunea proceselor	process interaction
resursă	resource
proces	process
renunțare (clienți)	reneging
blocare / înfometare	blocking / starvation
buffer (zonă tampon)	buffer
triaj	triage
geamăn digital	digital twin
simulare paralelă	parallel discrete-event simulation (PDES)
revenire în timp	rollback

Întrebări de autoevaluare

1. Care afirmație descrie corect o simulare cu evenimente discrete?

a) timpul simulat avansează mereu cu un pas fix Δt

b) starea se schimbă doar la anumite momente (evenimente), iar ceasul sare de la un eveniment la următorul

c) timpul simulat ia doar valori întregi

d) toate evenimentele sunt cunoscute înainte de începerea simulării

2. De ce este exactă simularea condusă de evenimente a bilei în cutie?

a) pentru că folosește un pas Δt foarte mic

b) pentru că folosește numere aleatoare

c) pentru că între ciocniri mișcarea este cunoscută exact, iar momentul ciocnirii se calculează direct

d) pentru că ignoră ciocnirile cu pereții

3. Ceasul arată t = 3. Lista evenimentelor viitoare conține: P1(5,2), S3(4,1), S4(6). Ce eveniment se procesează la pasul următor?

a) S3, iar ceasul devine 4,1

b) P1, iar ceasul devine 5,2

c) S4, iar ceasul devine 6

d) niciunul; ceasul avansează la 3,1

4. Care dintre următoarele este un eveniment **exogen** într-un model de bancă?

a) terminarea servirii unui client

b) eliberarea unui ghișeu

c) începerea servirii clientului din fața cozii

d) sosirea unui client nou

5. Cum se calculează corect numărul mediu de clienți $\bar{L}$ dintr-o simulare?

a) media valorilor lui N observate la fiecare eveniment

b) aria de sub N(t) împărțită la durata simulării

c) numărul de clienți la sfârșitul simulării

d) numărul total de sosiri împărțit la numărul de evenimente

6. Care este costul operațiilor de inserare și extragere a minimului într-un heap binar cu n evenimente?

a) O(1)

b) O(n)

c) O(log n)

d) O(n log n)

7. De ce punem un număr de ordine (seq) în tuplul fiecărui eveniment din heap?

a) ca evenimentele simultane să fie ordonate reproductibil și Python să nu compare funcțiile

b) ca să numărăm clienții

c) ca heap-ul să fie sortat complet

d) ca să accelerăm ceasul

8. În modelul M/M/1, un client sosește când serverul este **liber**. Ce evenimente programează acțiunea de sosire?

a) doar sosirea următoare

b) sosirea următoare și plecarea acestui client

c) doar plecarea acestui client

d) niciun eveniment

9. SimPy, în care viața unei entități este scrisă ca o funcție generator cu yield, folosește abordarea:

a) programării evenimentelor

b) scanării activităților (trei faze)

c) interacțiunii proceselor

d) simulării cu pas fix

10. În metoda în trei faze (Tocher), evenimentele C sunt:

a) evenimentele care au loc la momente cunoscute dinainte

b) evenimentele anulate

c) evenimentele de la sfârșitul simulării

d) evenimentele care au loc când devine adevărată o condiție (de ex. coadă nevidă și server liber)

11. Ce face yield env.timeout(5) într-un proces SimPy?

a) oprește programul 5 secunde reale

b) suspendă procesul și îl reia după 5 unități de timp simulat

c) creează 5 procese noi

d) mută ceasul înapoi cu 5 unități

12. Într-o linie de producție cu timpi de prelucrare **constanți**, ce efect are mărirea bufferelor dintre stații?

a) aproape niciun efect: linia atinge oricum ritmul stației celei mai lente

b) dublează throughput-ul

c) reduce throughput-ul

d) elimină gâtul de sticlă

13. Într-o coadă cu un server și priorități non-preemptive (timpi de servire identici), ce se întâmplă cu așteptarea medie peste **toți** clienții față de FIFO?

a) scade la jumătate

b) se dublează

c) rămâne aceeași: prioritățile redistribuie așteptarea între clase

d) devine zero pentru toți

14. În simularea paralelă optimistă (Time Warp), ce face un proces logic care primește un mesaj cu un marcaj de timp din trecutul lui?

a) ignoră mesajul

b) oprește toată simularea

c) așteaptă mesaje nule

d) revine la o stare salvată (rollback) și trimite anti-mesaje

15. Limbajul Simula 67, creat pentru simulare, este important în informatică pentru că:

a) a introdus clasele și obiectele – primul limbaj orientat pe obiecte

b) a fost primul limbaj compilat

c) a introdus programarea funcțională

d) a inventat heap-ul binar

16. Într-o simulare de 10 minute, N(t) = 1 timp de 3 min, N(t) = 2 timp de 2 min și N(t) = 0 în restul timpului. Care este numărul mediu de clienți $\bar{L}$?

(Răspuns numeric)

17. O bilă se află în punctul (2, 3) într-o cutie de 10 × 10, cu viteza (1; 0,5). După câte secunde are loc prima ciocnire cu un perete?

(Răspuns numeric)

18. Semaforul la cerere are latența a = 20 s și b = 5 s pe mașină. Pe roșu sosesc mașini la t = 10, 15 și 22 (nicio altă mașină). La ce moment revine semaforul pe roșu?

(Răspuns numeric)

19. La un ghișeu cu un server (FIFO) sosesc clienți la t = 0, 1 și 2, fiecare cu servirea de 2 minute. Care este așteptarea medie în coadă, în minute?

(Răspuns numeric)

20. La o unitate de urgențe sosesc în medie 12 pacienți pe oră, iar o consultație durează în medie 20 de minute. Câți medici sunt ocupați în medie (încărcarea oferită a)?

(Răspuns numeric)

Răspunsuri

1. b) – Între evenimente starea rămâne constantă, așa că simulatorul sare direct la următorul eveniment; timpul simulat este continuu.

2. c) – Momentul următoarei ciocniri se obține rezolvând o ecuație de gradul întâi; nu există eroare de discretizare.

3. a) – Se extrage mereu evenimentul cu cel mai mic moment: S3 la 4,1.

4. d) – Sosirile vin din afara sistemului și nu depind de starea lui; celelalte sunt consecințe ale evoluției sistemului (endogene).

5. b) – $\bar{L}$ este o medie ponderată în timp: fiecare valoare a lui N contează proporțional cu durata în care a fost valabilă.

6. c) – heappush și heappop parcurg cel mult înălțimea arborelui, adică O(log n).

7. a) – La momente egale, tuplurile se compară după al doilea element; contorul dă ordinea FIFO și evită compararea acțiunilor.

8. b) – Orice sosire programează sosirea următoare; dacă serverul e liber, servirea începe imediat și se programează și plecarea.

9. c) – Fiecare entitate este un proces; yield marchează punctele în care procesul așteaptă.

10. d) – Evenimentele B au momente cunoscute; evenimentele C sunt condiționate și se verifică în faza C.

11. b) – SimPy programează reluarea procesului peste 5 unități de timp simulat; între timp rulează alte procese.

12. a) – Bufferele absorb variabilitatea; fără variabilitate, linia merge deja în ritmul gâtului de sticlă.

13. c) – Clasa prioritară așteaptă mai puțin, cealaltă mai mult; media ponderată rămâne cea de la FIFO (legea conservării).

14. d) – Time Warp avansează optimist, iar la un mesaj întârziat revine înapoi în timp și anulează mesajele trimise greșit.

15. a) – Dahl și Nygaard au introdus clasele și obiectele pentru a descrie entitățile unei simulări.

16. 0,7 – $\bar{L} = (1\cdot3 + 2\cdot2 + 0\cdot5)/10 = 0{,}7$.

17. 8 – Pereții spre care se mișcă: dreapta după $(10 - 2)/1 = 8$ s, sus după $(10 - 3)/0{,}5 = 14$ s; minimul este 8.

18. 45 – Verde la 10 + 20 = 30; așteaptă 3 mașini, deci verdele durează $3\cdot5 = 15$ s: roșu la 45.

19. 1 – Clientul 1 începe la 0 (așteaptă 0), clientul 2 la 2 (așteaptă 1), clientul 3 la 4 (așteaptă 2); media = 1.

20. 4 – $a = \lambda\cdot s = 12$ pacienți/oră $\cdot$ (1/3) oră = 4 medici ocupați în medie (Little).

Bibliografie

- Prof. Cernazean – Modelling and Simulation, Lecture 7: Discrete Events Simulation (materiale de curs).
- Ș. Holban – Tehnici de modelare și simulare, cap. 2: modelarea bazată pe evenimente, graful activităților (materiale de curs).
- R. J. Gibbens – Computer Systems Modelling, University of Cambridge – simularea cu evenimente discrete.
- A. M. Law – Simulation Modeling and Analysis, ed. 5, McGraw-Hill, 2015.
- J. Banks, J. S. Carson, B. L. Nelson, D. M. Nicol – Discrete-Event System Simulation, ed. 5, Pearson, 2010.
- SimPy – Discrete event simulation for Python, documentația oficială (simpy.readthedocs.io).
- L. Schruben – Simulation modeling with event graphs, Communications of the ACM, 26(11), 1983.
- K. D. Tocher – The Art of Simulation, English Universities Press, 1963.
- O.-J. Dahl, K. Nygaard – SIMULA: an ALGOL-based simulation language, Communications of the ACM, 9(9), 1966.
- D. R. Jefferson – Virtual time, ACM Transactions on Programming Languages and Systems, 7(3), 1985.
- R. M. Fujimoto – Parallel and Distributed Simulation Systems, Wiley, 2000.
- Material realizat cu ajutorul Claude Fable 5.1 și Claude Opus 5.5.